

MANUAL DE PROGRAMACIÓN

NAHUEL VILLAFANE



HECHO EN ARGENTINA

MANUAL DE PROGRAMACIÓN

Python desde Cero

Repaso Completo - Todos los Temas

20 de mayo de 2026

Nahuel Villafañe

Basado en las clases y apuntes

Para estudiantes de programación

Índice

Variables y Tipos de Datos	4
Conceptos básicos	4
Reglas para nombrar variables	4
Operadores aritméticos	4
Ejercicios prácticos	5
Soluciones	5
Condicionales (if / elif / else)	6
Operadores de comparación	6
Estructura if-elif-else	6
Operadores lógicos	6
Ejercicios prácticos	7
Soluciones	7
Bucles (While y For)	9
Bucle <code>while</code>	9
Bucle <code>for</code> y <code>range()</code>	9
Break y Continue	9
Diferencia entre <code>while</code> y <code>for</code>	10
Ejercicios prácticos	10
Soluciones	10
Listas	12
Definición y acceso	12
Métodos importantes	12
Recorrer listas	13
Ejercicios prácticos	13
Soluciones	13
Diccionarios	15
Definición y acceso	15
Métodos importantes	15
Recorrer diccionarios	15
Ejercicios prácticos	16

Soluciones	16
Funciones	18
¿Qué es una función?	18
Estructura básica	18
Partes de una función	18
Función PURA vs IMPURA	18
Tipos de funciones según parámetros	19
Ejemplos clásicos de funciones	19
Cómo llamar a una función	20
Ámbito de las variables (scope)	21
Errores comunes en funciones	21
Ejercicios prácticos	22
Soluciones	22
Para seguir practicando (sin solución)	22
Módulo Random	24
Funciones principales	24
Ejemplo: Juego de adivinanza	24
Módulo os: El Explorador de Archivos en Python	25
¿Por qué necesitamos el módulo os?	25
Primeros pasos: importar y probar	25
¿Dónde estoy? - os.getcwd()	25
¿Qué hay dentro? - os.listdir()	26
Ejercicio 1: Mi primer explorador	26
Verificar qué es cada cosa: archivo o carpeta	26
Ejercicio 2: Clasificador automático	27
Crear carpetas - os.makedirs()	27
Ejercicio 3: Creador masivo de carpetas	27
Navegar entre carpetas - os.chdir()	27
Ejercicio 4: Navegador simple	28
Eliminar archivos - os.remove()	28
Renombrar - os.rename()	29
Información de archivos	29
Proyecto integrador: Mi propio explorador de archivos	29

Tabla resumen del módulo os	31
Ejercicios finales	31
Conclusión del capítulo	32
Proyectos Integradores	32
Sistema de gestión de alumnos	32
Carrito de compras	33
Juego del ahorcado	34
Errores Comunes y Cómo Evitarlos	35
Ejemplo de código con errores y su corrección	35
Ejemplos Rápidos (Chuleta)	36
Ejercicios Adicionales de Repaso	38
Nivel fácil	38
Nivel medio	38
Nivel difícil	38
Soluciones (seleccionadas)	38
Resumen de Sintaxis (Formulario)	40
Conclusión	41
Tabla de Operadores	43
Operadores aritméticos	43
Operadores de comparación	43
Operadores lógicos	43

Variables y Tipos de Datos

Conceptos básicos

Una **variable** es un espacio en memoria donde se guarda un valor. En Python, las variables se crean al asignarles un valor con el operador `=`.

```

1 # Tipos b\'asicos
2 nombre = "Nahuel"          # string (texto)
3 edad = 25                  # int (entero)
4 precio = 99.90             # float (decimal)
5 es_mayor = True            # bool (verdadero/falso)
6
7 # Input siempre devuelve string
8 dato = input("Ingrese algo: ")
9
10 # Convertir tipos
11 numero = int(input("Numero: "))    # string -> entero
12 decimal = float(input("Decimal: ")) # string -> float
13 texto = str(25)                    # n\'umero -> string

```

Listing 1: Tipos de datos básicos y conversión

Reglas para nombrar variables

- Pueden contener letras, números y guión bajo (`_`).
- No pueden empezar con números.
- No pueden ser palabras reservadas (`if`, `while`, `for`, etc.).
- Son sensibles a mayúsculas/minúsculas: `nombre` $\neq$ `Nombre`.

Operadores aritméticos

Operador	Símbolo	Ejemplo
Suma	+	$5 + 3 = 8$
Resta	-	$5 - 3 = 2$
Multiplicación	*	$5 * 3 = 15$
División	/	$6 / 3 = 2.0$
Módulo (resto)	%	$7 \% 3 = 1$
Potencia	**	$2 ** 3 = 8$
División entera	//	$7 // 3 = 2$

```

1 suma = 10 + 5          # 15
2 resta = 10 - 5         # 5
3 multi = 10 * 5         # 50
4 division = 10 / 5      # 2.0

```

```
5 modulo = 10 % 3          # 1
6 potencia = 2 ** 3        # 8
7 div_entera = 10 // 3     # 3
```

Listing 2: Ejemplos de operadores

Ejercicios prácticos

1. Crea variables con tu nombre, edad y altura. Muéstralas con `print()`.
2. Pide dos números al usuario y muestra su suma, resta, multiplicación y división.
3. Calcula el área de un rectángulo ($\text{base} \times \text{altura}$) con datos ingresados por el usuario.
4. Convierte grados Celsius a Fahrenheit: $F = C \times 9/5 + 32$.

Soluciones

```
1 # 1
2 nombre = "Nahuel"
3 edad = 25
4 altura = 1.75
5 print(nombre, edad, altura)
6
7 # 2
8 a = int(input("N1: "))
9 b = int(input("N2: "))
10 print("Suma:", a+b, "Resta:", a-b, "Multi:", a*b, "Div:", a/b)
11
12 # 3
13 base = float(input("Base: "))
14 altura = float(input("Altura: "))
15 print("\'Area:", base * altura)
16
17 # 4
18 c = float(input("Celsius: "))
19 f = c * 9/5 + 32
20 print(f"{c}°C = {f}°F")
```

Listing 3: Soluciones - Variables y tipos

Condicionales (if / elif / else)

Operadores de comparación

Operador	Significado	Ejemplo
==	igual a	if edad == 18:
!=	distinto de	if edad != 18:
>	mayor que	if edad > 18:
<	menor que	if edad < 18:
>=	mayor o igual	if edad >= 18:
<=	menor o igual	if edad <= 18:

Estructura if-elif-else

```

1 numero = int(input("N\úmero: "))
2
3 if numero > 0:
4     print("Positivo")
5 elif numero < 0:
6     print("Negativo")
7 else:
8     print("Cero")

```

Listing 4: Estructura condicional

Operadores lógicos

- and: ambas condiciones deben ser verdaderas.
- or: al menos una condición debe ser verdadera.
- not: invierte el valor de la condición.

```

1 # AND: ambas condiciones deben ser verdaderas
2 if edad >= 18 and tiene_dni:
3     print("Puede entrar")
4
5 # OR: al menos una condici\on debe ser verdadera
6 if dia == "sabado" or dia == "domingo":
7     print("Fin de semana")
8
9 # NOT: invierte el valor de la condici\on
10 if not lloviendo:
11     print("Salgo a pasear")

```

Listing 5: Ejemplos con operadores lógicos

Ejercicios prácticos

1. Determina si un número es par o impar.
2. Clasifica una nota: ≥ 9 Sobresaliente, ≥ 7 Notable, ≥ 5 Aprobado, < 5 Suspenso.
3. Pide un número del 1 al 7 y muestra el día de la semana correspondiente.
4. Calcula el IMC ($\text{peso}/\text{altura}^2$) e imprime: bajo peso ($< 18,5$), normal ($18,5-25$), sobrepeso ($25-30$), obesidad (> 30).

Soluciones

```
1 # 1
2 num = int(input("Nº número: "))
3 if num % 2 == 0:
4     print("Par")
5 else:
6     print("Impar")
7
8 # 2
9 nota = float(input("Nota: "))
10 if nota >= 9:
11     print("Sobresaliente")
12 elif nota >= 7:
13     print("Notable")
14 elif nota >= 5:
15     print("Aprobado")
16 else:
17     print("Suspenso")
18
19 # 3
20 dia = int(input("Nº número (1-7): "))
21 if dia == 1: print("Lunes")
22 elif dia == 2: print("Martes")
23 elif dia == 3: print("Miércoles")
24 elif dia == 4: print("Jueves")
25 elif dia == 5: print("Viernes")
26 elif dia == 6: print("Sábado")
27 elif dia == 7: print("Domingo")
28 else: print("Inválido")
29
30 # 4
31 peso = float(input("Peso (kg): "))
32 altura = float(input("Altura (m): "))
33 imc = peso / (altura**2)
34 if imc < 18.5: print("Bajo peso")
35 elif imc < 25: print("Normal")
36 elif imc < 30: print("Sobrepeso")
37 else: print("Obesidad")
```

Listing 6: Soluciones condicionales

Bucles (While y For)

Bucle while

Se repite mientras la condición sea verdadera. Es importante actualizar la variable de control para evitar bucles infinitos.

```
1 contador = 1
2 while contador <= 5:
3     print(contador)
4     contador = contador + 1    # ¡No olvidar!
```

Listing 7: While básico

Bucle for y range()

El for itera sobre una secuencia. `range(start, stop, step)` genera números.

```
1 # range(stop) -> 0,1,...,9
2 for i in range(10):
3     print(i)
4
5 # range(start, stop) -> 1..10
6 for i in range(1, 11):
7     print(i)
8
9 # range(start, stop, step) -> pares
10 for i in range(2, 11, 2):
11     print(i)
12
13 # Recorrer lista
14 frutas = ["manzana", "pera", "uva"]
15 for fruta in frutas:
16     print(fruta)
17
18 # Recorrer string
19 palabra = "Python"
20 for letra in palabra:
21     print(letra)
```

Listing 8: Ejemplos con for

Break y Continue

- `break`: sale del bucle inmediatamente.
- `continue`: salta a la siguiente iteración.

```

1 # Break: sale al encontrar 5
2 i = 1
3 while i <= 10:
4     if i == 5:
5         break
6     print(i)
7     i += 1
8
9 # Continue: salta el 5
10 i = 0
11 while i < 10:
12     i += 1
13     if i == 5:
14         continue
15     print(i)

```

Listing 9: Break y Continue

Diferencia entre while y for

WHILE	FOR
Necesita una condición	Es autónomo
Hay que poner un contador a mano	El contador es interno
Se usa cuando NO sabes cuántas vueltas	Se usa cuando SABES cuántas vueltas
while n < 5:	for i in range(5):
n = n + 1 (obligatorio)	No hace falta contador

Ejercicios prácticos

1. Muestra los números del 1 al 10 con **while**.
2. Suma los números del 1 al 100 con **for**.
3. Pide números hasta ingresar 0 y muestra la suma total.
4. Cuenta regresiva desde un número ingresado hasta 0.
5. Tabla de multiplicar de un número (usando **while** o **for**).

Soluciones

```

1 # 1
2 i = 1
3 while i <= 10:
4     print(i)

```

```
5     i += 1
6
7 # 2
8 suma = 0
9 for i in range(1, 101):
10     suma += i
11 print("Suma:", suma)
12
13 # 3
14 suma = 0
15 num = int(input("Nº número (0 termina): "))
16 while num != 0:
17     suma += num
18     num = int(input("Otro: "))
19 print("Total:", suma)
20
21 # 4
22 n = int(input("Nº número: "))
23 while n >= 0:
24     print(n)
25     n -= 1
26
27 # 5
28 n = int(input("Tabla: "))
29 for i in range(1, 11):
30     print(f"{n} x {i} = {n*i}")
```

Listing 10: Soluciones bucles

Listas

Definición y acceso

Una lista es una colección ordenada y mutable de elementos. Los índices comienzan en 0.

```

1 # Crear listas
2 lista = [1, 2, 3, 4, 5]
3 vacia = []
4 mixta = [1, "hola", True, 3.14]
5
6 # Acceder (\'indice empieza en 0)
7 print(lista[0])    # primer elemento: 1
8 print(lista[2])    # tercer elemento: 3
9 print(lista[-1])   # \'ultimo elemento: 5
10 print(lista[-2])   # pen\'ultimo elemento: 4
11
12 # Modificar
13 lista[2] = 99      # [1, 2, 99, 4, 5]
14
15 # Slicing (rebanadas)
16 print(lista[1:3])  # [2, 99]
17 print(lista[:3])   # [1, 2, 99]
18 print(lista[2:])   # [99, 4, 5]

```

Listing 11: Creación, acceso y slicing

Métodos importantes

Método	Descripción
append(x)	Agrega x al final de la lista.
insert(i, x)	Inserta x en la posición i.
remove(x)	Elimina la primera aparición de x.
pop(i)	Elimina y devuelve el elemento en i (por defecto el último).
sort()	Ordena la lista de forma ascendente.
sort(reverse=True)	Ordena de forma descendente.
reverse()	Invierte el orden de los elementos.
clear()	Vacía la lista.
count(x)	Cuenta cuántas veces aparece x.
index(x)	Devuelve la posición de la primera aparición de x.
len(lista)	Devuelve la cantidad de elementos.

```

1 lista = [3, 1, 4, 1, 5]
2 lista.append(9)          # [3,1,4,1,5,9]
3 lista.insert(2, 99)      # [3,1,99,4,1,5,9]
4 lista.remove(1)          # elimina el primer 1
5 lista.pop()              # elimina el 9

```

```
6 lista.pop(2)           # elimina el elemento en \'indice 2
7 lista.sort()           # [1,3,4,5,99]
8 lista.reverse()        # [99,5,4,3,1]
9 lista.clear()          # []
```

Listing 12: Ejemplos de métodos

Recorrer listas

```
1 # Forma 1: recorrer los elementos
2 for elemento in lista:
3     print(elemento)
4
5 # Forma 2: recorrer por \'indice
6 for i in range(len(lista)):
7     print(i, lista[i])
8
9 # Forma 3: con enumerate (\'indice y valor)
10 for i, elemento in enumerate(lista):
11     print(f"Posici\'on {i}: {elemento}")
```

Listing 13: Formas de recorrer una lista

Ejercicios prácticos

1. Crea una lista de 5 colores y muéstralos.
2. Pide 3 números al usuario, guárdalos en una lista con `append()` y luego muestra la suma.
3. Dada una lista de números, encuentra el número más grande.
4. Separa los números pares e impares de una lista en dos listas nuevas.
5. Pide nombres hasta que el usuario ingrese "fin" luego muestra la lista ordenada.

Soluciones

```
1 # 1
2 colores = ["rojo", "azul", "verde", "amarillo", "negro"]
3 for c in colores:
4     print(c)
5
6 # 2
7 nums = []
8 for i in range(3):
9     nums.append(int(input("N\'umero: ")))
10 print("Suma:", sum(nums))
11
```

```
12 # 3
13 nums = [5, 2, 8, 1, 9]
14 mayor = nums[0]
15 for n in nums:
16     if n > mayor:
17         mayor = n
18 print("Mayor:", mayor)
19
20 # 4
21 pares = []
22 impares = []
23 for n in [1,2,3,4,5,6]:
24     if n % 2 == 0:
25         pares.append(n)
26     else:
27         impares.append(n)
28 print("Pares:", pares, "Impares:", impares)
29
30 # 5
31 nombres = []
32 while True:
33     nom = input("Nombre (fin termina): ")
34     if nom == "fin":
35         break
36     nombres.append(nom)
37 nombres.sort()
38 print(nombres)
```

Listing 14: Soluciones listas

Diccionarios

Definición y acceso

Un diccionario almacena pares `clave: valor`. Las claves son únicas.

```

1 persona = {"nombre": "Nahuel", "edad": 25}
2 print(persona["nombre"])           # Nahuel
3 persona["ciudad"] = "Buenos Aires" # Agregar
4 persona["edad"] = 26                # Modificar
5 del persona["ciudad"]               # Eliminar

```

Listing 15: Creación y acceso básico

Métodos importantes

Método	Descripción
<code>keys()</code>	Devuelve todas las claves.
<code>values()</code>	Devuelve todos los valores.
<code>items()</code>	Devuelve pares (clave, valor).
<code>get(clave, default)</code>	Devuelve el valor asociado a <code>clave</code> , o <code>default</code> si no existe (por defecto <code>None</code>).
<code>pop(clave)</code>	Elimina la clave y devuelve su valor.
<code>update(otro_dic)</code>	Actualiza el diccionario con las claves y valores de otro diccionario.
<code>clear()</code>	Vacía el diccionario.
<code>in</code>	Verifica si una clave existe: <code>"nombre" in persona</code> .

Recorrer diccionarios

```

1 persona = {"nombre": "Nahuel", "edad": 25, "ciudad": "Bs As"}
2
3 # Recorrer claves
4 for clave in persona:
5     print(clave, ":", persona[clave])
6
7 # Recorrer valores
8 for valor in persona.values():
9     print(valor)
10
11 # Recorrer clave-valor
12 for clave, valor in persona.items():
13     print(f"{clave}: {valor}")

```

Listing 16: Formas de recorrer un diccionario

Ejercicios prácticos

1. Crea un diccionario con 3 personas (nombre, edad, ciudad).
2. Agrega una nueva persona al diccionario.
3. Usa `.get()` para obtener la edad de una persona que no existe, con mensaje personalizado.
4. Recorre el diccionario y muestra solo los nombres de las personas mayores de 18 años.
5. Crea una agenda telefónica (nombre: teléfono) con funciones para agregar, buscar y eliminar contactos.

Soluciones

```
1 # 1
2 personas = {
3     "Ana": {"edad": 22, "ciudad": "CABA"},
4     "Luis": {"edad": 25, "ciudad": "C\ordoba"}
5 }
6 # 2
7 personas["Marta"] = {"edad": 21, "ciudad": "Rosario"}
8
9 # 3
10 edad = personas.get("Pedro", "No existe")
11 print(edad)
12
13 # 4
14 for nombre, datos in personas.items():
15     if datos["edad"] > 18:
16         print(nombre)
17
18 # 5
19 agenda = {}
20 def agregar():
21     nom = input("Nombre: ")
22     tel = input("Tel\efono: ")
23     agenda[nom] = tel
24 def buscar():
25     nom = input("Nombre: ")
26     print(agenda.get(nom, "No existe"))
27 def eliminar():
28     nom = input("Nombre: ")
29     if nom in agenda:
30         del agenda[nom]
31         print("Eliminado")
32     else:
33         print("No existe")
```

```
34 # El men\'u se puede implementar con un while que llame a estas  
    funciones
```

Listing 17: Soluciones diccionarios

Funciones

¿Qué es una función?

Una función es un bloque de código que realiza una tarea específica y puede reutilizarse. Es como una receta que recibe ingredientes (parámetros), los procesa y devuelve un resultado.

Estructura básica

```

1 def nombre(parametro1, parametro2):
2     """
3     Documentación de la función (opcional)
4     """
5     # Código
6     resultado = parametro1 + parametro2
7     return resultado

```

Listing 18: Estructura de una función

Partes de una función

Parte	Qué hace	Ejemplo
def	Palabra clave para definir función	def sumar(a, b):
nombre	Cómo se llama la función	sumar
parámetros	Datos que recibe la función	(a, b)
:	Dos puntos al final de la línea	def sumar(a, b):
return	Devuelve un resultado (opcional)	return a + b

Función PURA vs IMPURA

```

1 # =====
2 # FUNCIÓN PURA
3 # - Solo usa return
4 # - No imprime nada
5 # - No modifica variables externas
6 # - Mismo resultado siempre para los mismos parámetros
7 # =====
8 def suma_pura(a, b):
9     return a + b
10

```

```
11 resultado = suma_pura(5, 3) # resultado = 8
12
13 # =====
14 # FUNCION IMPURA
15 # - Imprime (print) o modifica algo externo
16 # - Tiene "efectos secundarios"
17 # =====
18 def suma_impura(a, b):
19     print(a + b) # Esto es un efecto secundario
20     # No tiene return, devuelve None
21
22 suma_impura(5, 3) # imprime 8, pero no devuelve nada
```

Listing 19: Diferencia entre función pura e impura

Tipos de funciones según parámetros

```
1 # 1. Sin parámetros
2 def saludar():
3     return "Hola"
4
5 # 2. Con un parámetro
6 def doble(numero):
7     return numero * 2
8
9 # 3. Con varios parámetros
10 def suma(a, b, c):
11     return a + b + c
12
13 # 4. Con parámetros por defecto
14 def saludar_persona(nombre="Invitado"):
15     return f"Hola {nombre}"
16
17 # 5. Con número variable de parámetros (*args)
18 def sumar_todos(*numeros):
19     total = 0
20     for n in numeros:
21         total = total + n
22     return total
23
24 # 6. Con parámetros nombrados (**kwargs)
25 def mostrar_datos(**datos):
26     for clave, valor in datos.items():
27         print(f"{clave}: {valor}")
```

Listing 20: Funciones con diferentes parámetros

Ejemplos clásicos de funciones

```
1 # Par o impar
2 def es_par(numero):
3     return numero % 2 == 0
4
5 # N\úmero primo
6 def es_primo(n):
7     if n < 2:
8         return False
9     for divisor in range(2, n):
10         if n % divisor == 0:
11             return False
12     return True
13
14 # Factorial
15 def factorial(n):
16     resultado = 1
17     for i in range(1, n + 1):
18         resultado = resultado * i
19     return resultado
20
21 # Palabra m\as larga de una lista
22 def palabra_mas_larga(lista):
23     mas_larga = lista[0]
24     for palabra in lista:
25         if len(palabra) > len(mas_larga):
26             mas_larga = palabra
27     return mas_larga
28
29 # Suma de lista
30 def suma_lista(lista):
31     total = 0
32     for elemento in lista:
33         total = total + elemento
34     return total
35
36 # Promedio de lista
37 def promedio(lista):
38     if len(lista) == 0:
39         return 0
40     return suma_lista(lista) / len(lista)
```

Listing 21: Funciones útiles ya definidas

Cómo llamar a una función

```
1 # Definir la funci\on
2 def suma(a, b):
3     return a + b
4
5 # Llamar a la funci\on
```

```

6 resultado = suma(5, 3)    # resultado = 8
7 print(resultado)
8
9 # Tambi\en se puede usar directamente
10 print(suma(10, 20))      # 30
11
12 # Guardar el resultado en una variable
13 x = suma(2, 2)           # x = 4
14 y = suma(x, 3)           # y = 7

```

Listing 22: Llamadas a funciones

Ámbito de las variables (scope)

```

1 # Variable GLOBAL (fuera de la funci\on)
2 nombre_global = "Nahuel"
3
4 def mi_funcion():
5     # Variable LOCAL (dentro de la funci\on)
6     nombre_local = "Juan"
7
8     # Para modificar una variable global, usar 'global'
9     global nombre_global
10    nombre_global = "Carlos"
11
12    print(nombre_local)    # Juan
13    print(nombre_global)  # Carlos
14
15 mi_funcion()
16 print(nombre_global)     # Carlos (cambi\o)
17 # print(nombre_local)    # ERROR: nombre_local no existe fuera

```

Listing 23: Variables locales vs globales

Errores comunes en funciones

Error	Ejemplo INCORRECTO	Corrección CORRECTA
Olvidar : Olvidar return Return dentro de un for antes de terminar Parámetro que no se usa Variable no definida Confundir = y ==	def suma(a,b) def suma(a,b): a+b return dentro del bucle def f(lista): (y no la usa) usar x sin definir if x = 5:	def suma(a,b): return a+b return fuera del for Sacar el parámetro definir x antes if x == 5:

Ejercicios prácticos

1. Función que reciba un número y devuelva su doble.
2. Función que reciba una lista y devuelva su promedio.
3. Función que determine si un número es primo.
4. Función que cuente las vocales de una cadena.
5. Función que reciba una lista de palabras y devuelva la más larga.

Soluciones

```
1 # 1
2 def doble(n):
3     return n * 2
4
5 # 2
6 def promedio(lista):
7     return sum(lista) / len(lista) if lista else 0
8
9 # 3
10 def es_primo(n):
11     if n < 2: return False
12     for i in range(2, int(n**0.5)+1):
13         if n % i == 0:
14             return False
15     return True
16
17 # 4
18 def contar_vocales(texto):
19     vocales = "aeiouAEIOU"
20     return sum(1 for letra in texto if letra in vocales)
21
22 # 5
23 def palabra_mas_larga(lista):
24     return max(lista, key=len) if lista else ""
```

Listing 24: Soluciones funciones

Para seguir practicando (sin solución)

1. Función que reciba un nombre y devuelva "Hola- nombre".
2. Función que reciba un número y devuelva True si es par.
3. Función que reciba una lista y devuelva una lista con todos los elementos multiplicados por 2.
4. Función que reciba una palabra y cuente sus vocales.

5. Función que calcule el factorial de un número.
6. Función que reciba dos números y devuelva su promedio.

Módulo Random

Funciones principales

```
1 import random
2
3 # Entero aleatorio entre a y b
4 numero = random.randint(1, 10)
5
6 # Decimal aleatorio entre 0 y 1
7 decimal = random.random()
8
9 # Elegir un elemento aleatorio de una lista
10 opciones = ["piedra", "papel", "tijera"]
11 cpu = random.choice(opciones)
12
13 # Elegir varios elementos aleatorios (con repetición)
14 seleccion = random.choices(opciones, k=3)
15
16 # Elegir varios elementos aleatorios (sin repetición)
17 seleccion = random.sample(opciones, k=2)
18
19 # Mezclar una lista aleatoriamente
20 cartas = [1, 2, 3, 4, 5]
21 random.shuffle(cartas)
```

Listing 25: Uso del módulo random

Ejemplo: Juego de adivinanza

```
1 import random
2
3 secreto = random.randint(1, 20)
4 intentos = 3
5
6 while intentos > 0:
7     num = int(input("Adivina el número (1-20): "))
8     if num == secreto:
9         print("¡Ganaste!")
10        break
11    elif num < secreto:
12        print("Más alto")
13    else:
14        print("Más bajo")
15    intentos = intentos - 1
16 else:
17    print(f"Perdiste. El número era {secreto}")
```

Listing 26: Adivina el número

Módulo os: El Explorador de Archivos en Python

¿Por qué necesitamos el módulo os?

Hasta ahora, todos nuestros programas trabajaban solo con datos dentro del propio programa: números, listas, diccionarios. Pero, ¿qué pasa cuando queremos que nuestro programa interactúe con el sistema operativo? Por ejemplo:

- Leer el contenido de una carpeta.
- Crear carpetas o archivos nuevos.
- Saber en qué directorio estamos parados.
- Renombrar o eliminar archivos.

Para todo eso necesitamos el módulo **os** (de **operating system**). Es como si le diéramos a Python un permiso especial para *hablar* con el sistema operativo y hacer cosas más allá de la memoria del programa.

Importante: El módulo **os** funciona en Windows, Linux y Mac, pero algunas rutas o comandos pueden cambiar ligeramente. A lo largo del capítulo te avisaré cuando haya diferencias.

Primeros pasos: importar y probar

Para usar **os**, lo primero es importarlo:

```
1 import os
```

¿Dónde estoy? - os.getcwd()

Cuando ejecutas un programa, Python *se para* en alguna carpeta de tu computadora. Para saber cuál es:

```
1 import os
2 ruta_actual = os.getcwd() # Get Current Working Directory
3 print(f"Estoy en: {ruta_actual}")
```

Ejecuta ese código. Te mostrará algo como:

Estoy en: C:\Users\TuNombre\Documentos

o en Linux/Mac:

Estoy en: /home/tunombre/Documentos

¿Qué hay dentro? - os.listdir()

Para ver todo el contenido de la carpeta actual (archivos y subcarpetas):

```
1 contenido = os.listdir()
2 print(contenido)
```

Esto devuelve una **lista** de strings con los nombres. Puedes recorrerla:

```
1 for elemento in os.listdir():
2     print(elemento)
```

Ejercicio 1: Mi primer explorador

Escribe un programa que muestre:

1. La ruta actual.
2. La cantidad de elementos dentro (usa `len()`).
3. El nombre de cada elemento en una línea aparte.

```
1 # Solucion
2 import os
3 print("Ruta:", os.getcwd())
4 lista = os.listdir()
5 print("Cantidad de elementos:", len(lista))
6 for e in lista:
7     print("-", e)
```

Verificar qué es cada cosa: archivo o carpeta

No todo lo que ves en una carpeta es un archivo. Algunos son carpetas (directorios). Para diferenciarlos usamos:

- `os.path.isfile(ruta)` devuelve True si es un archivo.
- `os.path.isdir(ruta)` devuelve True si es una carpeta.

```
1 import os
2 for elemento in os.listdir():
3     if os.path.isdir(elemento):
4         print(f"[CARPETA] {elemento}")
5     elif os.path.isfile(elemento):
6         print(f"[ARCHIVO] {elemento}")
```

Ejercicio 2: Clasificador automático

Haz un programa que recorra la carpeta actual y cuente cuántos archivos y cuántas carpetas hay.

```
1 # Solucion
2 import os
3 archivos = 0
4 carpetas = 0
5 for e in os.listdir():
6     if os.path.isfile(e):
7         archivos += 1
8     elif os.path.isdir(e):
9         carpetas += 1
10 print(f"Archivos: {archivos}")
11 print(f"Carpetas: {carpetas}")
```

Crear carpetas - os.makedirs()

Para crear una carpeta nueva:

```
1 os.makedirs("mi_nueva_carpeta")
```

¿Y si quieres crear varias carpetas dentro de otras? También puedes:

```
1 os.makedirs("nivel1/nivel2/nivel3")
```

Eso creará tres carpetas anidadas. Si alguna ya existe, Python mostrará un error. Para evitarlo, puedes usar `exist_ok=True`:

```
1 os.makedirs("carpeta", exist_ok=True) # No da error si ya existe
```

Ejercicio 3: Creador masivo de carpetas

Pide al usuario un nombre base y cuántas carpetas quiere crear. Usa un bucle `for` para crearlas con números: `base_1`, `base_2`, etc.

```
1 # Solucion
2 import os
3 base = input("Nombre base: ")
4 cantidad = int(input("Cuántas carpetas?: "))
5 for i in range(cantidad):
6     nombre = f"{base}_{i+1}"
7     os.makedirs(nombre, exist_ok=True)
8     print(f"Creada: {nombre}")
```

Navegar entre carpetas - os.chdir()

Para *moverse* a otra carpeta (como si hicieras doble clic en el explorador):

```
1 os.chdir("nombre_de_carpeta") # Change Directory
```

Después de eso, `os.getcwd()` mostrará la nueva ubicación.

Cuidado: Si la carpeta no existe, Python dará error. Siempre verifica antes con `os.path.exists()`.

```
1 if os.path.exists("musica"):
2     os.chdir("musica")
3 else:
4     print("No existe la carpeta 'musica'")
```

Ejercicio 4: Navegador simple

Haz un programa que:

1. Muestre la ruta actual.
2. Pida al usuario el nombre de una carpeta.
3. Si existe, que se mueva allí y muestre la nueva ruta.
4. Si no existe, que muestre un mensaje de error.

```
1 # Solucion
2 import os
3 print("Estas en:", os.getcwd())
4 carpeta = input("A que carpeta quieres entrar? ")
5 if os.path.isdir(carpeta):
6     os.chdir(carpeta)
7     print("Ahora estas en:", os.getcwd())
8 else:
9     print("No existe esa carpeta")
```

Eliminar archivos - `os.remove()`

Para borrar un archivo (¡Cuidado! no va a la papelera, se borra para siempre):

```
1 os.remove("archivo_inutil.txt")
```

Siempre verifica que existe antes:

```
1 if os.path.exists("archivo.txt"):
2     os.remove("archivo.txt")
3     print("Archivo eliminado")
4 else:
5     print("No existe")
```

Renombrar - `os.rename()`

Para cambiar el nombre de un archivo o carpeta:

```
1 os.rename("viejo_nombre.txt", "nuevo_nombre.txt")
```

Información de archivos

- `os.path.getsize(ruta)` â€Š tamaño en bytes.
- `os.path.abspath(ruta)` â€Š ruta completa absoluta.

```
1 if os.path.isfile("foto.jpg"):
2     tama~no = os.path.getsize("foto.jpg")
3     ruta_abs = os.path.abspath("foto.jpg")
4     print(f'"{ruta_abs}" pesa {tama~no} bytes')
```

Proyecto integrador: Mi propio explorador de archivos

Vamos a construir un programa que funcione como un pequeño explorador de archivos desde la consola. Tendrá un menú con estas opciones:

1. Mostrar directorio actual.
2. Listar carpetas (solo carpetas).
3. Listar archivos (solo archivos).
4. Cambiar de carpeta.
5. Crear carpeta.
6. Eliminar archivo.
7. Salir.

```
1 import os
2
3 def mostrar_menu():
4     print("\n" + "="*40)
5     print("    EXPLORADOR DE ARCHIVOS")
6     print("="*40)
7     print("1. Donde estoy?")
8     print("2. Ver carpetas")
9     print("3. Ver archivos")
10    print("4. Entrar a carpeta")
11    print("5. Crear carpeta")
12    print("6. Eliminar archivo")
13    print("7. Salir")
```

```
14     print("-"*40)
15
16 def main():
17     while True:
18         mostrar_menu()
19         opcion = input("Elige una opcion: ")
20
21         if opcion == "1":
22             print(f"\nEstas en: {os.getcwd()}")
23
24         elif opcion == "2":
25             print("\n--- CARPETAS ---")
26             for e in os.listdir():
27                 if os.path.isdir(e):
28                     print(f"    [DIR]    {e}")
29
30         elif opcion == "3":
31             print("\n--- ARCHIVOS ---")
32             for e in os.listdir():
33                 if os.path.isfile(e):
34                     print(f"    [FILE] {e}")
35
36         elif opcion == "4":
37             nombre = input("Nombre de la carpeta: ")
38             if os.path.isdir(nombre):
39                 os.chdir(nombre)
40                 print(f"Ahora estas en: {os.getcwd()}")
41             else:
42                 print("No existe esa carpeta")
43
44         elif opcion == "5":
45             nombre = input("Nombre de la nueva carpeta: ")
46             try:
47                 os.makedirs(nombre, exist_ok=False)
48                 print(f"Carpeta '{nombre}' creada")
49             except FileExistsError:
50                 print("Ya existe una carpeta con ese nombre")
51
52         elif opcion == "6":
53             nombre = input("Nombre del archivo a eliminar: ")
54             if os.path.isfile(nombre):
55                 os.remove(nombre)
56                 print("Archivo eliminado")
57             else:
58                 print("No existe ese archivo")
59
60         elif opcion == "7":
61             print("Â¡Hasta luego!")
62             break
63
64     else:
```

```

65         print("Opcion no valida")
66
67 if __name__ == "__main__":
68     main()

```

Tabla resumen del módulo os

Función	Qué hace
<code>os.getcwd()</code>	Obtener directorio actual
<code>os.listdir()</code>	Listar contenido de la carpeta actual
<code>os.chdir(ruta)</code>	Cambiar a otra carpeta
<code>os.makedirs(ruta)</code>	Crear carpeta/s (puede crear varias de una vez)
<code>os.remove(archivo)</code>	Eliminar un archivo (¡Cuidado!)
<code>os.rename(original, nuevo)</code>	Renombrar archivo o carpeta
<code>os.path.exists(ruta)</code>	Preguntar si existe
<code>os.path.isfile(ruta)</code>	Preguntar si es archivo
<code>os.path.isdir(ruta)</code>	Preguntar si es carpeta
<code>os.path.getsize(ruta)</code>	Obtener tamaño en bytes
<code>os.path.abspath(ruta)</code>	Obtener ruta absoluta
<code>os.path.join(a,b)</code>	Unir rutas de forma segura según el SO

Cuadro 8: Las funciones más útiles del módulo os

Ejercicios finales

1. **Nivel fácil:** Crea un programa que te diga cuánto espacio ocupa la carpeta actual (suma los tamaños de todos los archivos, no entres en subcarpetas).
2. **Nivel medio:** Crea un *buscador de archivos* que pida al usuario una extensión (ej: `.txt`, `.py`) y muestre todos los archivos con esa extensión en la carpeta actual.
3. **Nivel difícil:** Crea un programa que recorra **todas** las subcarpetas (usando `os.walk()`) y encuentre el archivo más pesado de todo el árbol.

```

1 # Solucion ejercicio 1 (facil)
2 import os
3 total = 0
4 for e in os.listdir():
5     if os.path.isfile(e):
6         total += os.path.getsize(e)
7 print(f"Peso total: {total} bytes")
8
9 # Solucion ejercicio 2 (medio)
10 import os
11 ext = input("Extension (ej: .py): ")
12 if not ext.startswith("."):
13     ext = "." + ext

```

```
14 encontrados = []
15 for e in os.listdir():
16     if os.path.isfile(e) and e.endswith(ext):
17         encontrados.append(e)
18 print(f"Archivos {ext}:", encontrados)
```

Conclusión del capítulo

El módulo `os` es una de esas herramientas que al principio parecen complicadas, pero una vez que las entiendes, las usas todo el tiempo. Con él puedes:

- Hacer programas que organicen tus descargas automáticamente.
- Crear backups de carpetas enteras.
- Renombrar cientos de archivos con un solo bucle.
- Construir tus propios exploradores de archivos.

Lo más importante que quiero que recuerdes es: **siempre verifica antes de actuar**. Antes de borrar, pregunta si existe. Antes de entrar a una carpeta, confirma que sea carpeta. Así evitarás errores y, más importante, pérdida de datos.

Ahora ya tienes todas las herramientas para que tus programas interactúen con el sistema operativo. ¡A programar se ha dicho!

Proyectos Integradores

Sistema de gestión de alumnos

```
1 alumnos = []
2 notas = {}
3
4 def agregar_alumno():
5     nombre = input("Nombre: ")
6     alumnos.append(nombre)
7     notas[nombre] = []
8     print("Agregado")
9
10 def agregar_nota():
11     nombre = input("Alumno: ")
12     if nombre in alumnos:
13         nota = int(input("Nota: "))
14         notas[nombre].append(nota)
15     else:
16         print("No existe")
17
18 def mostrar_promedios():
```

```
19     for alumno in notas:
20         if notas[alumno]:
21             prom = sum(notas[alumno]) / len(notas[alumno])
22             print(f"{alumno}: {prom:.2f}")
23
24 # Menú principal (ejemplo)
25 while True:
26     print("1. Agregar alumno")
27     print("2. Agregar nota")
28     print("3. Mostrar promedios")
29     print("4. Salir")
30     op = input("Opción: ")
31     if op == "1":
32         agregar_alumno()
33     elif op == "2":
34         agregar_nota()
35     elif op == "3":
36         mostrar_promedios()
37     elif op == "4":
38         break
```

Listing 27: Gestión de alumnos y notas

Carrito de compras

```
1 carrito = []
2
3 def agregar():
4     nombre = input("Producto: ")
5     precio = float(input("Precio: "))
6     cantidad = int(input("Cantidad: "))
7     carrito.append({"nombre": nombre, "precio": precio, "cantidad":
8                     cantidad})
9
10 def mostrar():
11     for item in carrito:
12         subtotal = item["precio"] * item["cantidad"]
13         print(f"{item['nombre']} x{item['cantidad']} = ${subtotal
14               :.2f}")
15
16 def total():
17     total = sum(item["precio"] * item["cantidad"] for item in
18                 carrito)
19     print(f"Total: ${total:.2f}")
20
21 # Menú similar al anterior
```

Listing 28: Carrito con diccionarios

Juego del ahorcado

```
1 import random
2 palabras = ["python", "programacion", "algoritmo"]
3 secreta = random.choice(palabras)
4 adivinadas = []
5 intentos = 6
6
7 while intentos > 0:
8     palabra_mostrada = "".join([letra if letra in adivinadas else "
9     _" for letra in secreta])
10    print(palabra_mostrada)
11    letra = input("Letra: ").lower()
12    if letra in secreta:
13        adivinadas.append(letra)
14        if set(adivinadas) == set(secreta):
15            print("¡Ganaste!")
16            break
17    else:
18        intentos -= 1
19        print(f"Fallaste. Te quedan {intentos} intentos")
20 else:
21    print(f"Perdiste. La palabra era {secreta}")
```

Listing 29: Estructura del ahorcado

Errores Comunes y Cómo Evitarlos

Error	Ejemplo INCORRECTO	Corrección CORRECTA
Falta : = en lugar de == No convertir input Índice fuera de rango Bucle infinito Olvidar return Parámetro que no se usa Variable global sin global División por cero Concatenar tipos	if x >5 if x = 5 x = input() lista[10] while x != 0: def suma(a,b): a+b def f(lista): modificar variable global a / b .edad: - edad	if x >5: if x == 5 x = int(input()) if len(lista) >10: agregar x = x - 1 return a+b sacar el parámetro usar global x if b != 0: f.edad: edad"

Ejemplo de código con errores y su corrección

```

1 # C\ODIGO CON ERRORES
2 def calcular_promedio(notas)
3     suma = 0
4     for i in range(len(notas))
5         suma = suma + notas(i)
6     promedio = suma / len(notas)
7     print("Promedio: " + promedio)
8
9 mis_notas = [10, 8, 9]
10 calcular_promedio(mis_notas)

```

Listing 30: Código con errores

```

1 # C\ODIGO CORREGIDO
2 def calcular_promedio(notas):           # Agregar :
3     suma = 0
4     for i in range(len(notas)):         # Agregar :
5         suma = suma + notas[i]          # Cambiar () por []
6     promedio = suma / len(notas)
7     print(f"Promedio: {promedio}")      # Usar f-string
8
9 mis_notas = [10, 8, 9]
10 calcular_promedio(mis_notas)

```

Listing 31: Código corregido

Ejemplos Rápidos (Chuleta)

```
1 # Par o impar
2 if numero % 2 == 0:
3     print("Par")
4 else:
5     print("Impar")
6
7 # Sumar lista
8 suma = 0
9 for n in lista:
10     suma = suma + n
11
12 # Palabra m\'as larga
13 mas_larga = lista[0]
14 for palabra in lista:
15     if len(palabra) > len(mas_larga):
16         mas_larga = palabra
17
18 # N\'umero primo
19 def es_primo(n):
20     if n < 2:
21         return False
22     for divisor in range(2, n):
23         if n % divisor == 0:
24             return False
25     return True
26
27 # Contar vocales
28 def contar_vocales(texto):
29     vocales = 0
30     for letra in texto.lower():
31         if letra in "aeiou":
32             vocales = vocales + 1
33     return vocales
34
35 # Invertir lista
36 def invertir_lista(lista):
37     return lista[::-1]
38
39 # Factorial
40 def factorial(n):
41     resultado = 1
42     for i in range(1, n + 1):
43         resultado = resultado * i
44     return resultado
45
46 # Promedio de lista
47 def promedio(lista):
48     return sum(lista) / len(lista) if lista else 0
```

Listing 32: Códigos útiles para copiar y pegar

Ejercicios Adicionales de Repaso

Nivel fácil

1. Escribe un programa que pida el nombre y lo muestre 10 veces (usando `for`).
2. Pide un número y muestra su tabla de multiplicar del 1 al 10.
3. Crea una función que devuelva `True` si un número es par.
4. Pide dos números y muestra el mayor.
5. Crea una lista con los primeros 10 números naturales e invierte el orden.

Nivel medio

1. Pide números hasta ingresar 0 y cuenta cuántos fueron positivos.
2. Crea un diccionario con los meses y sus días (Enero:31, etc.). Pide un mes y muestra sus días.
3. Función que reciba una lista y devuelva una nueva lista solo con los números positivos.
4. Simula el juego "Piedra, papel o tijera" contra la computadora (mejor de 3).
5. Crea una función que reciba una frase y devuelva la palabra más larga.

Nivel difícil

1. Crea un programa que genere una contraseña aleatoria de 8 caracteres (mayúsculas, minúsculas, números, símbolos).
2. Escribe una función que reciba un número y devuelva su factorial (usando `while`).
3. Crea un sistema de login con 3 intentos y un diccionario de usuarios.
4. Simula un combate por turnos (jugador vs monstruo) con ataque, defensa y curación.
5. Crea un programa que permita gestionar un inventario (agregar, eliminar, mostrar, buscar).

Soluciones (seleccionadas)

```
1 # Piedra, papel o tijera (mejor de 3)
2 import random
3 puntos_j = 0
4 puntos_c = 0
5 while puntos_j < 3 and puntos_c < 3:
6     jugador = input("Piedra, papel o tijera: ").lower()
```

```

7     cpu = random.choice(["piedra", "papel", "tijera"])
8     if jugador == cpu:
9         print("Empate")
10    elif (jugador == "piedra" and cpu == "tijera") or \
11          (jugador == "papel" and cpu == "piedra") or \
12          (jugador == "tijera" and cpu == "papel"):
13        puntos_j += 1
14        print("Ganaste ronda")
15    else:
16        puntos_c += 1
17        print("Perdiste ronda")
18        print(f"Marcador: {puntos_j} - {puntos_c}")
19    print("¿Ganaste el juego!" if puntos_j == 3 else "Perdiste el
    juego!")
20
21    # Generador de contraseñas
22    import random, string
23    def generar_password(long=8):
24        chars = string.ascii_letters + string.digits + string.
25        punctuation
26        return ''.join(random.choice(chars) for _ in range(long))
27    print(generar_password())
28
29    # Combate por turnos
30    vida_j = 100
31    vida_m = 100
32    while vida_j > 0 and vida_m > 0:
33        accion = input("Atacar / Defender / Curar: ").lower()
34        dano_j = random.randint(10,30)
35        dano_m = random.randint(10,25)
36        if accion == "atacar":
37            vida_m -= dano_j
38            vida_j -= dano_m
39        elif accion == "defender":
40            vida_j -= dano_m // 2
41        elif accion == "curar":
42            vida_j += 20
43            if vida_j > 100: vida_j = 100
44        print(f"Vida J: {vida_j} | Vida M: {vida_m}")
45    print("¿Ganaste!" if vida_m <= 0 else "Game Over")

```

Listing 33: Soluciones de ejercicios adicionales

Resumen de Sintaxis (Formulario)

Esta sección es un formulario rápido de consulta. Aquí encontrarás la sintaxis exacta de cada estructura de Python, sin explicaciones largas.

Estructuras básicas

Concepto	Sintaxis	Ejemplo
If simple	<code>if condicion:</code>	<code>if x > 0:</code>
If-else	<code>if cond:</code> <code>else:</code>	<code>if x > 0: print("+") else: print("-")</code>
If-elif-else	<code>if cond1:</code> <code>elif cond2:</code> <code>else:</code>	<code>if x>0: print("+") elif x<0: print("-")</code>
Bucle while	<code>while condicion:</code>	<code>while x < 10: x += 1</code>
Bucle for	<code>for var in secuencia:</code>	<code>for i in range(5):</code>
range()	<code>range(stop)</code> / <code>range(start,stop)</code> / <code>range(start,stop,step)</code>	<code>range(10), range(1,11), range(2,11,2)</code>
Break	<code>break</code>	<code>if x==5: break</code>
Continue	<code>continue</code>	<code>if x==5: continue</code>

Listas

Crear lista	<code>lista = [elementos]</code>	<code>nums = [1,2,3]</code>
Agregar al final	<code>lista.append(x)</code>	<code>nums.append(4)</code>
Insertar	<code>lista.insert(pos, x)</code>	<code>nums.insert(1, 99)</code>
Eliminar por valor	<code>lista.remove(x)</code>	<code>nums.remove(99)</code>
Eliminar por índice	<code>lista.pop(pos)</code>	<code>nums.pop(2)</code>
Ordenar	<code>lista.sort()</code>	<code>nums.sort()</code>
Orden inverso	<code>lista.sort(reverse=True)</code>	<code>nums.sort(reverse=True)</code>
Dar vuelta	<code>lista.reverse()</code>	<code>nums.reverse()</code>
Vaciar	<code>lista.clear()</code>	<code>nums.clear()</code>
Largo	<code>len(lista)</code>	<code>len(nums)</code>

Diccionarios

Crear diccionario	<code>dic = {clave: valor}</code>	<code>persona = {"nombre": "Ana"}</code>
Acceder	<code>dic[clave]</code>	<code>persona["nombre"]</code>
Acceder seguro	<code>dic.get(clave, default)</code>	<code>persona.get("edad", 0)</code>
Agregar/Modificar	<code>dic[clave] = valor</code>	<code>persona["edad"] = 25</code>
Eliminar	<code>del dic[clave]</code>	<code>del persona["edad"]</code>
Claves	<code>dic.keys()</code>	<code>list(persona.keys())</code>

Valores	<code>dic.values()</code>	<code>list(persona.values())</code>
Pares clave-valor	<code>dic.items()</code>	<code>for k,v in persona.items():</code>

Funciones

Definir	<code>def nombre(params):</code>	<code>def suma(a,b):</code>
Return	<code>return valor</code>	<code>return a+b</code>
Sin return	(imprime o modifica)	<code>def saludar(): print("Hola")</code>
Parámetros por defecto	<code>def f(param=valor)</code>	<code>def saludo(nombre="Invitado")</code>
Args variables	<code>def f(*args)</code>	<code>def sumar_todos(*nums): return sum(nums)</code>
Llamar	<code>resultado = nombre(args)</code>	<code>total = suma(5,3)</code>

Módulos

Importar módulo	<code>import modulo</code>	<code>import random</code>
Importar función específica	<code>from modulo import funcion</code>	<code>from random import randint</code>
Importar con alias	<code>import modulo as alias</code>	<code>import random as rnd</code>

Módulo random (funciones clave)

Entero aleatorio	<code>random.randint(a,b)</code>	<code>random.randint(1,10)</code>
Decimal 0-1	<code>random.random()</code>	<code>random.random()</code>
Elegir elemento	<code>random.choice(lista)</code>	<code>random.choice(["A","B"])</code>
Mezclar lista	<code>random.shuffle(lista)</code>	<code>random.shuffle(cartas)</code>

Módulo os (funciones clave)

Directorio actual	<code>os.getcwd()</code>	<code>ruta = os.getcwd()</code>
Listar contenido	<code>os.listdir()</code>	<code>archivos = os.listdir()</code>
Cambiar directorio	<code>os.chdir(ruta)</code>	<code>os.chdir("carpeta")</code>
Crear carpeta/s	<code>os.makedirs(ruta)</code>	<code>os.makedirs("a/b/c")</code>
Eliminar archivo	<code>os.remove(archivo)</code>	<code>os.remove("datos.txt")</code>
Renombrar	<code>os.rename(orig, nuevo)</code>	<code>os.rename("a.txt","b.txt")</code>

Conclusión

Este manual ha cubierto los conceptos fundamentales de Python: variables, tipos de datos, condicionales, bucles, listas, diccionarios, funciones, módulos y proyectos integradores. La

mejor manera de aprender es practicando. No te quedes solo con la teoría: escribe código, equivócate, depura y vuelve a intentar.

¡Felicitaciones por llegar hasta aquí! Ahora tienes las herramientas para seguir explorando el mundo de la programación.

"La mejor manera de aprender Python es escribiendo código, equivocándose y volviendo a intentarlo."

Tabla de Operadores

Operadores aritméticos

Operador	Símbolo	Ejemplo
Suma	+	$5 + 3 = 8$
Resta	-	$5 - 3 = 2$
Multiplicación	*	$5 * 3 = 15$
División	/	$6 / 3 = 2.0$
Módulo	%	$7 \% 3 = 1$
Potencia	**	$2 ** 3 = 8$
División entera	//	$7 // 3 = 2$

Operadores de comparación

Operador	Significado	Ejemplo
==	igual a	$5 == 5 \rightarrow \text{True}$
!=	distinto de	$5 != 3 \rightarrow \text{True}$
>	mayor que	$5 > 3 \rightarrow \text{True}$
<	menor que	$5 < 3 \rightarrow \text{False}$
>=	mayor o igual	$5 >= 5 \rightarrow \text{True}$
<=	menor o igual	$5 <= 3 \rightarrow \text{False}$

Operadores lógicos

Operador	Significado	Ejemplo
and	ambas deben ser True	$\text{True and False} \rightarrow \text{False}$
or	al menos una debe ser True	$\text{True or False} \rightarrow \text{True}$
not	invierte el valor	$\text{not True} \rightarrow \text{False}$